

FIT1051 Assignment Two (Semester One, 2025)

Submission deadline: Friday 2nd May 11:55pm AEST via email

Weight: 15% of your overall unit mark

Late Penalty: 5% mark deduction per day

Instructions: Below are the tasks that you need to complete for Assessment 2. Your work and your submission should be independent. Please download the IntelliJ project folder and unzip it. This will provide you with code you need to complete the assignment task.

This assessment is worth 15% of the unit total. There is no interview component for this assessment, therefore the marks below make up 100% of the overall assessment mark.

- Task 1 is worth 25 marks
- Task 2 is worth 25 marks
- Task 3 is worth 25 marks
- Task 4 is worth 25 marks

Submission: Complete the assignment in a single document that includes a title page with your name and student ID. You must submit your assignment as a PDF document to the Assessment 2 link on the Moodle Assessments page. **MAKE SURE YOU DOWNLOAD FROM MOODLE AFTER THAT TO CHECK IT IS THE RIGHT SUBMISSION!**

Academic Integrity: Please be reminded of the academic integrity standards that are expected of you at Monash, which were mentioned in Week 1. You should work alone and ask the unit staff for help if needed. Do not post your work in public forums or send your work to anyone. Do not copy/paste text or code from other sources and present it as your own — this includes use of generative AI tools. Breaching these academic integrity requirements can incur serious penalties.

Please note, as per university requirements, similarity detection will be used across all student submissions. You may be asked for a short interview if there are any concerns regarding similarity or the use of Generative AI.

Instructions: This task requires you to analyse the Java program that has been provided in the accompanying ZIP file. The java program is the Generative AI solution to a previous FIT1051 assignment.

In the ZIP file is a folder that can be opened in IntelliJ. It has 2 Java class files. You must read through the code (and can run it if you wish) to understand how it has been designed and how it works.

You must submit a written report that covers these four requirements. All have equal weighting.

Section A: Describing how the solution successfully meets all requirements of the scenario given [approx. 500 words]

Write a summary of how well the given solution successfully meets all requirements for the given scenario. This refers to the code structure as well as functionality. You should directly address the key expected functionality in the given scenario.

Section B: Describing the Approach to Object-Oriented Design [approx. 500 words]

Write a summary of the Object-Oriented approach to the design of this program. You should describe the choice of classes, attributes, methods, and any other relevant Java design. You will be assessed on how well you identify the different design features, and your articulation on why they were used, given the program overview (on the following page).

Section C: Program Testing [approx. 500 words]

Prepare an appropriate testing plan for the program (including a table of sample input and expected output), given the provided program overview and the logical design of the code (e.g loops and decision statements). Describe your rationale for the different tests that you describe. You will be assessed on how well you identify all relevant tests required and your articulation of their rationale.

Section D: Meeting the FIT1051 Coding Guidelines [approx. 500 words]

Write a summary of how the program conforms to the published FIT1051 Coding Standards. They can be found here:

<https://edstem.org/au/courses/20568/lessons/70310/slides/469138>. You will be assessed on how well you apply these coding standards to the provided program.

Program Overview:

You need to make a simple user registration application that allows users to register for a system by entering their email address and chosen password. Once a user has registered, they can then log in to the system to change their email or password, or delete their account.

A **RegisteredUser** class should have three appropriately typed fields, **userID**, **email** and **password**. The user can change the email address or password associated with their account at any time, however their userID should remain the same for as long as they are registered in the system. Each user's userID should be unique, and userIDs should be assigned in increasing numerical order with each new registration, starting from 1.

The **RegisteredUser** class should have:

- A default constructor with sensible default values, and a non-default constructor with sensible parameters.
- Getter and setter methods for fields as appropriate.
- A toString method that returns the details of the **RegisteredUser**, formatted appropriately.

There is also a **Assignment2Driver** class, that does the following, when the program is run:

- Initialises an empty ArrayList of **RegisteredUsers**.
- Outputs a main menu, which allows the user to select from the following options:
 1. Register a new user.
 2. Login as an existing user.
 3. Quit the program.

The program will continue to run until Option 3 is selected.

If Option 1 is selected, the user should be prompted to enter the email address and password they want to register with. A new **RegisteredUser** object containing their details should be added to the list of users. The user should be informed that they have been registered successfully, and their registration details (including userID) should be output to the screen, before returning them to the main menu. Recall that userIDs should be uniquely assigned, in increasing order starting at 1.

If Option 2 is selected, the user should be prompted to login by entering their email address and password. They should be allowed three attempts to enter this correctly. If after the third attempt, no match for their email/password has been found, an appropriate error message should be printed and the user should be returned to the main menu. If, on the other hand, the user logs in successfully, they should be presented with a new menu, with the following options:

1. Print user details, which outputs all the information associated with that user in the system.
2. Change email address, which prompts the user for a new email address and updates their details in the system.
3. Change password, which prompts the user for a new password and updates their details in the system.
4. Delete account, which if selected, will prompt the user for confirmation, then remove their details from the ArrayList and return them to the main menu.
5. Log out, which returns the user to the main menu.

The system should also include the following validation checks:

A method called **validateEmail** in the Assignment2Driver class. This method should take an email address as its parameter, and return a boolean value. It should check the provided email address to ensure it contains the character '@', preceded by at least one character, and followed by at least one character. If the email address meets these criteria, the method should return true. Otherwise, it should return false. For example, the strings "hello@world", "1@1", and "student123@monash.edu" should all return **true**, but the strings "helloworld", "rebecca@", and "@monash.edu" should all return **false**.

A method called **validatePassword** in the Assignment2Driver class. This method should take a password as its parameter, and return a boolean value. It should check the provided password to ensure it:

- is at least 8 characters in length,
- contains at least one number, and
- contains a mix of upper and lowercase characters.

If it meets these three conditions, it should return **true**, otherwise it should return **false**.